Chapter 5 Monte Carlo Methods

核心思想：什么是蒙特卡洛方法？

蒙特卡洛方法不需要对环境有完全的了解（不需要知道所有的状态转移概率），它只需要**经验 (experience)**。

💡 **通俗理解：** 假设你想知道一个不规则骰子掷出“6”的概率。

- **动态规划 (Dynamic Programming) 的做法：** 拿卡尺量骰子的重心、计算空气动力学、推导物理公式，最后算出精确概率。（需要环境的完全已知模型）
- **蒙特卡洛 (Monte Carlo) 的做法：** 啥也不算，直接把这个骰子掷 10000 次，记录出现“6”的次数，求个平均值。（只需要不断尝试的经验）

在强化学习里，蒙特卡洛方法就是通过不断地在环境里“跑测验”（采样），然后把得到的奖励**求平均**，以此来评估某个状态或动作的价值。

理论基石：为什么“求平均”是靠谱的？

大数定律 (Law of Large Numbers, LLN)

X ，$X1$ ，… ，Xn为独立同分布的实值随机变量（抽取的具体样本值），当样本数量 n 趋向于无穷大时，样本均值会(almost surely)收敛到真实的期望值（$E[X]$ ）。

$$\frac{1}{n}\sum_{i=1}^{n} X_i \xrightarrow{a.s.} \mathbb{E}[X], \quad when \;\; n \to \infty.$$

打个比方，“瞎蒙的次数多了，就接近真理了”。你抛硬币的次数越多，正面朝上的比例就越无限接近 50%。这保证了蒙特卡洛方法**最终一定能算对。**

在上述独立同分布条件下，对于任意实值函数f ，$f(X1)$ ，... ，$f(Xn)$也是相互独立同分布的随机变量，因此大数定律同样适用于函数值。

$$\frac{1}{n}\sum_{i=1}^{n} f(X_i) \xrightarrow{a.s.} \mathbb{E}[f(X)], \quad \text{when } n \to \infty.$$

中心极限定理 (Central Limit Theorem, CLT)
大数定律只说了“最终会收敛”，但没说“收敛得多快”。中心极限定理告诉我们，样本均值的误差分布服从正态分布，且方差与样本数 n 成反比。

假设 $X_1, \ldots, X_n$ 是独立同分布的实值随机变量，且对于每个 i，有 $\mathbb{E}[X_i] = \mu$ 和 $\mathrm{Var}[X_i] = \sigma^2$。设 $\bar{X}_n \triangleq \frac{1}{n}(X_1 + \cdots + X_n)$ 为样本均值，则随机变量 $\sqrt{n}(\bar{X}n - \mu)$ 依分布收敛（Convergence in Distribution）（概率分布图（直方图）长得越来越像另一个概率分布图）于正态分布 $\mathcal{N}(0, \sigma^2)$

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2), \quad as \ n \to \infty.$$

根据中心极限定理，随着 n 的增大，$\bar{X}_n$ 的分布趋近于均值为 μ、方差为 σ^2/n 的正态分布。换句话说，样本均值的方差与 n 成反比。数学推导：

$$\mathrm{Var}(\frac{X_1 + \cdots + X_n}{n}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$$

收敛速度：注意左边的 $\sqrt{n}$。这意味着误差是以 $1/\sqrt{n}$ 的速度缩小的。如果你想让误差减小到原来的 $1/10$，你需要将样本量 n 增加 100 倍。

中心极限定理 (CLT) 可以应用于函数值，因为 $f(X)$ 也是一个随机变量。鉴于随机变量 X 在上下文中很明确，我们令 $I_n(f) \triangleq \frac{1}{n}\sum i = 1^n f(X_i)$ 为函数值的样本均值。则：

$$\sqrt{n}\,(I_n(f)-\mu_f)\xrightarrow{d}\mathcal{N}(0,\sigma_f^2),\quad \text{as } n\to\infty,$$

其中 μ_f 和 σ_f 分别是 $f(X)$ 的期望和方差。

注意 $I_n(f)$ 的方差为 σ_f^2/n。为了将此方差降低到一定水平，例如 σ_ϵ^2，样本数量至少为 $\sigma_f^2/\sigma_\epsilon^2$。

也就是说，大数定律告诉你“坚持就是胜利”，中心极限定理告诉你“进度条走到哪了”。它说明：你采样的样本越多（n 越大），你的评估结果就越稳定（方差越小，误差越小）。为了让误差缩小到原来的 $\frac{1}{2}$，你需要增加 4 倍的采样量。

重要性采样 (Importance Sampling)

这时，就可能存在一个问题：有时候，我们想评估的“目标分布 p”（比如一个极其危险的自动驾驶策略）**很难或不能直接去采样**。那怎么办？咋搞？

重要性采样提供了一种近似期望值的替代方法。具体而言，重要性采样是一种用于评估期望值的蒙特卡洛方法。其基本思路是从提议分布q（比如人类驾驶员的日常数据）中抽取样本，并通过**重新加权(reweight)**这些样本来计算期望值：q 的结果折算成 p 的结果

具体来说，考虑空间 χ 上的两个分布，其概率密度函数分别为 p 和 q。假设 $p \prec q$，即 $p(x)>0 \Rightarrow q(x)>0, \forall x \in \mathcal{X}$。注意，这里的意思是绝对连续性，即$q$（辅助分布）的“覆盖范围”必须包含 p（目标分布）的所有可能取值范围。那么，我们有：

$$\mathbb{E}_p[f(X)]=\int_{\mathcal{X}} f(x)p(x)\nu(dx)=\int_{\mathcal{X}} f(x)\frac{p(x)}{q(x)}q(x)\nu(dx)=\mathbb{E}_q[w(x)\cdot f(x)],$$

其中 $w(x)=p(x)/q(x)$ 称为 x 的重要性权重。

▼ 符号解析

$\nu(dx)$ 被称为**测度，**它代表你在这个空间里“数数”或“量大小”的方式。**对于连续变量：** 就是我们熟悉的 dx，代表在数轴上的微小长度。**对于离散变量：** 代表在某个点上的计数。它是为了让公式更通用。无论你的数据是连续的（如身高）还是离散的（如掷骰子），这个公式都成立。

$p(x)$ **(目标分布)：** 这是“理想情况”。你想知道在分布 p 下的平均值（期望）。但遗憾的是，你可能无法直接从 p 里面拿到数据。

$q(x)$ **(提议分布/采样分布)：** 这是“现实工具”。它是你实际用来产生随机数的分布。它是你“借”来用的路标。

$f(x)$ **(目标函数)：** 这是你想考察的“指标”。比如 X 是气温，$f(X)$ 可能是“当天的耗电量”。**注意：$f(x)$ 本身在左右两边是一模一样的**，我们改变的只是计算它平均值的方式。

$I_{q,n}(f)$ 是**样本均值的估算量**：
- I：代表 Integral（积分）或 Importance（重要性）。
- **下标** q：提醒你，这些数据是从分布 q 里抽出来的。
- **下标** n：提醒你，你一共抽了 n 个样本。
- **含义**：它就是你在电脑上算出来的那个**最终数值结果**。它是对真实期望 $\mathbb{E}_p[f(X)]$ 的一个近似。

*公式解析：想求 $f(X)$ 在分布 p 下的期望。1.写出期望的定义 - 根据连续型随机变量期望的定义，它是函数值乘以概率密度函数的积分；2.引入辅助分布 - 只要 $q(x) \neq 0$，我们就可以在积分号里同时乘以 $q(x)$ 再除以 $q(x)$； 3.重新识别为 q 下的期望 - 积分式变成了“某个函数”乘以“密度函数 $q(x)$”的形式

$$\int_{\mathcal{X}} \left[f(x) \frac{p(x)}{q(x)} \right] q(x) \nu(dx) = \mathbb{E}_q[w(x) \cdot f(x)]$$

当你用 q 代替 p 去观察世界时，视野是歪曲的。$w(x)$ 的作用就是纠正。如果某个点 x，在 p 中很重要（密度高）但在 q 中被低估了（密度低），那么 $w(x) = p/q$ 就会很大，用来**放大**这个点的贡献。反之，如果 q 过分强调了某个点，权重就会变小，用来**缩小**它的影响。

上述公式将将关于 p 的期望，转化成关于 q 的期望。打个比方：

原任务 (w.r.t. p)： 你想算全班同学的平均身高，但你只有**全校**同学的数据。

转化后 (w.r.t. q)： 你去全校随机抽人（按 q 采样），但每抽到一个人，你要问他是不是那个班的

- 如果他是那个班的，你就给他很高的权重；
- 如果他不是那个班的，你就给他 0 权重。

因此，我们可以用从 q 中抽取的样本来近似 $\mathbb{E}_p[f(X)]$。特别地，设 $x_1, \ldots, x_n$ 是来自 q 的独立同分布样本，则

$$\mathbb{E}_p[f(X)] \simeq I_{q,n}(f) \triangleq \frac{1}{n}\sum_{i=1}^{n} w(x_i)\cdot f(x_i) = \frac{1}{n}\sum_{i=1}^{n} \frac{p(x_i)}{q(x_i)}\cdot f(x_i).$$

*公式解析：在计算机实现中，我们没法算无穷积分，只能采样（离散群）

由于我们已经证明了 $\mathbb{E}_p[f(X)] = \mathbb{E}_q[w(x)f(x)]$，那么：
1. 我们从分布 q 中抽取 n 个独立的样本 $x_1, x_2, \ldots, x_n$。
2. 根据大数定律，**期望** $\simeq$ **样本均值**。
3. 所以，我们将这 n 个样本对应的函数值与其"重要性权重" $w(x_i)$ 相乘，再取平均：

$$I_{q,n}(f) = \frac{1}{n}\sum_{i=1}^{n} w(x_i)f(x_i)$$

根据大数定律，我们有（样本数很大，无限接近p分布的期望）

$$I_{q,n}(f) \xrightarrow{a.s.} \mathbb{E}_p[f(X)], \quad \text{when } n \to \infty.$$

自归一化权重 (Self-normalized weights)

在实际应用中，归一化常数 p 和 q 未知且难以计算的情况并不少见：

$$p(x) = \frac{1}{Z_p}\tilde{p}(x), \quad q(x) = \frac{1}{Z_q}\tilde{q}(x),$$

其中 Z_p 和 Z_q 未知。对于这种情况，我们可以计算非归一化重要性 $\tilde{w}(x) = \tilde{p}(x)/\tilde{q}(x)$.这是**未归一化的权重**。它只利用了我们知道的形状信息

在概率论中，一个合格的概率密度函数必须满足：全空间积分等于 1
$\tilde{p}(x)$ **和** $\tilde{q}(x)$ **（未归一化函数）：** 这是你实际拿到的函数，比如 e^{-x^2}。它描述了分布的形状，但积分可能不等于 1。

打个比方： $\tilde{p}$ 像是还没切开的蛋糕形状，Z_p 则是蛋糕的总重量。除以 Z_p 后，你得到的才是每块蛋糕占总量的比例（概率）

然后我们可以使用自归一化权重来近似期望 $E_p[f(\hat{X})]$，如下所示

$$\mathbb{E}_p[f(X)] \approx \sum_{i=1}^{n} (\text{归一化后的权重}) \times f(x_i)$$

$$E_p[f(X)] \simeq \frac{1}{n}\sum_{i=1}^{n} \tilde{w}_i' f(x_i), \quad \text{with } \tilde{w}_i' = \frac{\tilde{w}(x_i)}{\sum_{j=1}^{n} \tilde{w}(x_j)}.$$

分母是所有样本权重之和，

i：代表当前正在计算的第 i 个样本。

j：是求和符号里的索引，代表把这 n 个样本全部扫一遍。

意义：这个操作强行让所有的权重加起来等于 1。通过这种方式，**原本未知的 Z_p 和 Z_q 会在比例中互相抵消掉**。

这里，$(\tilde{w}i')_{1:n}$z 被称为自归一化权重。

通俗的比喻来说，这就像发年终奖。老板不知道全行业标准的年终奖基数是多少（不知道 Z），他只知道员工 A 的贡献是员工 B 的 2 倍（知道相对比例 $\tilde{w}$）。于是老板直接拿出现有的 10 万块钱奖金池，按照 2:1 的比例把钱分给 A 和 B。这就是“自内部归一化”。

蒙特卡洛价值估计 (Monte Carlo Estimation of Values)

状态值估计（**Estimation of state values**） 我们首先考虑蒙特卡洛方法，用于估计给定策略 π 的状态值函数。 即不知道游戏内部机制（环境模型）我们怎么评估“某个状态有多好”或者“某个动作有多

**注意：本章假设基础过程为分段式*episodic*，即该过程由若干分段构成，且每个分段均保证在有限时间步内终止。在此设定下，可在有限时间范围内完整模拟一个分段。这是前提，也就是说：蒙特卡洛方法必须在有明确“结局”的任务中使用（比如一局王者荣耀，或者走迷宫走到终点）。因为我们需要拿到最终的“总回报 (return)”，然后再倒推回来看之前走过的每一步有多大贡献。

评估状态价值：$V^\pi(s)$

具体而言，对于状态$s \in S$，若要近似估计$V^\pi(s)$，可直接对访问该状态s后的收益取平均值

方法有两种实现方式（假设我们模拟了若干个场景）

首次访问 (First-visit MC)： 在一局游戏中，如果我多次路过同一个地点 s，我**只拿第一次**路过 s 后的最终得分来算平均得到$V^\pi(s)$。

每次访问 (Every-visit MC)： 在一局游戏中，**每一次**路过地点 s，我都单独记录一次它后续的得分，统统拿来算平均。

结论：根据大数定律，当访问 s 的次数趋于无穷大时，首次访问和每次访问蒙特卡罗方法都收敛于 $V^\pi(s)$。

动作值估计（Estimation of action values） 注意，对于给定的策略 π，动作值函数 Q^π 可以表示为：

$$Q^\pi(s,a) = \sum_{r,s'} p(s',r|s,a)\left(r + \gamma V^\pi(s')\right).$$

然而，此计算需要完全了解 $p(s',r|s,a)$。如果没有这样的模型（“没有上帝视角”），仅凭状态值是不够的。必须显式地估计每个动作的值，才能找到最佳策略。

这可以使用蒙特卡罗方法进行估计，即计算每个状态-动作对 (s,a) 之后收益的平均值。这里，访问 (s,a)意味着访问状态 s 并执行动作 a。

上述估计动作值的方法会遇到一个问题——许多状态-动作对可能永远不会被访问。这种情况在策略是确定性策略时尤为常见。打个比方当前的策略是个“死脑筋”（比如打游戏只会一直按攻击键），那“防御键”的价值永远得不到评估（没人按过，没有样本）

克服这一困难的一种方法是，在每个样本回合开始时，从一个分布中同时抽取初始状态 s 和后续动作 a，该分布中每个状态-动作对都具有非零概率。这保证了在无限回合的极限情况下，所有状态-动作对都会被访问。这种方法称为探索性起始(Exploring Start)。

就像每次开局时，**随机强制**把玩家扔到一个状态，并**随机强制**他做一个动作（比如开局就被扔进毒圈并强制按防御）。这样只要玩的局数无限多，所有“状态-动作”的组合就都能被体验到，保证了评估的全面

蒙特卡洛控制 (Monte Carlo Control)

通过蒙特卡洛估计，我们能够推导出一种不依赖于环境完整知识的强化学习方法。其基本思路是遵循广义策略迭代（Generalized Policy Iteration, GPI）范式，同时利用蒙特卡洛方法对价值函数进行估计。

蒙特卡罗控制的基本算法（Basic algorithm of Monte Carlo control）

该算法从给定的策略 π_0 开始，随后交替执行以下两个步骤：

1）蒙特卡洛策略评估，用于计算每个状态-动作对(s，a)的 Q^{π_k} 值；

2）策略改进，可通过使策略对当前价值函数具有贪婪性来实现。(贪心算法)：

$$\pi_{k+1}(s) \leftarrow \operatorname*{argmax}_{a \in \mathcal{A}} Q^{\pi_k}(s,a).$$

我们将该算法（及其下文介绍的变体）称为**蒙特卡洛控制**。对于上述迭代过程，我们对每个状态 $s \in S$ 有：

$$Q^{\pi_k}(s,\pi_{k+1}(s)) = Q^{\pi_k}(s,\operatorname*{argmax}_a Q^{\pi_k}(s,a)) = \max_{a\in\mathcal{A}} Q^{\pi_k}(s,a) \geq Q^{\pi_k}(s,\pi_k(s)) \geq V^{\pi_k}(s).$$

在状态 s 时，如果我们听从新策略 π_{k+1}（也就是挑最大 Q 值的动作去执行），得到的回报 $\max_a Q^{\pi_k}(s,a)$，**必然大于或等于**我们听从旧策略 π_k 得到的平均回报。
既然每一步都不比原来差，那么累加起来的总价值也一定更高（公式 5.12 的结论）。这就保证了算法**绝对不会越学越退步**。

π_k: 当前的旧策略。

π_{k+1}: 采取贪心算法后得到的新策略。

$Q^{\pi_k}(s,a)$: 在旧策略下，状态 s 采取动作 a 的价值。

$V^{\pi_k}(s)$: 在旧策略下，状态 s 的整体价值。

根据策略改进定理（*policy improvement theorem*），我们得到

$$V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s), \quad \forall s \in \mathcal{S}. \tag{5.12}$$

因此，上述算法通过迭代改进策略。

探索性启动的蒙特卡罗控制（Monte Carlo control with exploring starts）

该算法存在一个根本性缺陷：每次策略评估都需要无限次采样（infinitely many episodes）来精确计算价值函数，这显然不切实际。解决这一问题的思路是放弃在每次迭代中完成策略评估的尝试。我们可以借鉴动态规划中的价值迭代原理，将价值函数逐步逼近 Q^{π_k} 。

具体来说，每次策略评估只需采样一个回合，用其更新价值函数Q，并立刻改进一次策略。

为了让这个过程极度高效，这里引入了两个非常经典的算法优化技巧：

- 倒序计算累计回报 G_t

$$G_t \leftarrow \gamma G_{t+1} + R_{t+1}.$$

如果正向计算，每一时刻 t 的回报 G_t 都要把后面的奖励一直加到回合结束，时间复杂度是 $O(T^2)$。但如果我们从游戏结局**从后往前倒推 (from tail to front)**，当前时刻的回报 G_t 就等于下一时刻的回报 G_{t+1} 打个折扣 (γ)，加上刚拿到的奖励 R_{t+1}。这样只需遍历一次，复杂度直接降为 $O(T)$。

- 增量更新平均值

算平均值通常需要把所有历史数据存进一个大数组，然后求和除以总数。随着游戏局数增加，内存会爆炸

$$\begin{aligned} C(s,a) &\leftarrow C(s,a) + 1, \\ Q(s,a) &\leftarrow Q(s,a) + \frac{1}{C(s,a)}(G - Q(s,a)). \end{aligned}$$

新估计值 = 老估计值 + 步长 * (实际目标值 - 老估计值)

这里 $\frac{1}{C(s,a)}$ 就是步长，$(G - Q(s,a))$ 是误差。通过这个公式，我们**不需要保存历史数据**，只需要记住当前的 Q 值和被访问过的次数 C，就能完美算出平均值。极致节省内存。

Algorithm 5.1 Monte Carlo control with exploring starts

```
for each s ∈ S do                                          ▷ Initialization
    Initialize C(s, a) = 0, ∀s ∈ S, a ∈ A(s)
    Initialize Q(s, a) ∈ ℝ, ∀s ∈ S, a ∈ A(s)
    Initialize π(s) = argmax_a Q(s, a), ∀s ∈ S
end for
for each episode do
    Choose s_0 ∈ S, a_0 ∈ A(s) randomly                    ▷ Exploring starts
    Sample an episode following π: (s_0, a_0, r_1, ..., s_{T-1}, a_{T-1}, r_T)
    Let g_T ← 0
    for each t = T-1, T-2, ..., 0 do                       ▷ Update values and the policy
        Let g_t ← γ g_{t+1} + r_{t+1}
        Let C(s_t, a_t) ← C(s_t, a_t) + 1
        Let Q(s_t, a_t) ← Q(s_t, a_t) + (1/C(s_t, a_t)) (g_t − Q(s_t, a_t))
        if Q(s_t, a_t) > Q(s_t, π(s_t)) then               ▷ Keep the policy greedy
            Let π(s_t) ← a_t
        end if
    end for
end for
return π and Q
```

算法解释：

1. 初始化 (Initialization)

相当于在内存里建了三个大字典（哈希表）：

- `C(s, a)`：计数器。记录在状态 s 下执行动作 a 的次数，初始全为 0。
- `Q(s, a)`：价值表。记录在状态 s 下执行动作 a 的平均得分，随便初始化。
- $\pi(s)$：策略表。记录在状态 s 下"最该做哪个动作"，初始直接选 Q 值最大的。

2. 收集数据 (Episode Loop)

- **Exploring starts：** 强行把智能体随机塞到任意一个状态 s_0，并强行让它执行一个随机动作 a_0。
- **Sample an episode：** 接下来，就让它按照当前的策略 π 玩到底，直到游戏结束（比如打完一整局或者走到迷宫出口）。把沿途产生的所有数据按时间线记录成一个数组：$(s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T)$。

3. 更新价值与策略 (Update Loop - 倒序遍历)

这是算法的灵魂所在，**从游戏的最后一步往前倒推**（`for each t = T-1 ... 0`）：

- **算总分：** $g_t \leftarrow \gamma g_{t+1} + r_{t+1}$。当前时刻的总分，等于下一时刻的总分打个折($\gamma$)，加上刚刚拿到的奖励。
- **计数与更新 Q 值：** 访问次数 `C` 加 1。然后用经典的增量公式更新 Q 值，把最新的总分 g_t 揉进平均分里。
- **贪心更新策略：** `if Q(st, at) > Q(st, pi(st))`。如果在刚才的更新中，当前动作 a_t 的分数，超过了原本策略字典里存的那个"最佳动作"，那就立刻"见异思迁"，把策略表里的 $\pi(s_t)$ 替换成 a_t。

非探索启动的蒙特卡罗控制（Monte Carlo control without exploring starts）

探索性起始的目的是确保所有状态-动作值都被访问。然而，这种方法效率低下且没有必要。下文我们将介绍一种改进设计，该设计无需探索性起始，但仍能确保所有状态-动作对都被访问。

核心思路是采用软策略，即对所有$s \in S$和$a \in A(s)$都满足 $\pi(a|s) > 0$，但逐步使其趋近于最优确定性策略。特别地，我们仅考虑 ε -软策略，即对于任意$s \in A(s)$都满足$\pi(a|s) \geq \varepsilon/|A(s)|$的策略，并尝试从中寻找最优解。

在此条件下，我们设计了一种 ε -贪婪策略，该策略类似于为多臂老虎机引入的 ε -贪婪方法，但适用于马尔可夫决策过程场景。具体而言，该方法以小概率 ε 随机选择动作，随后每个非贪婪动作以概率 $\varepsilon/|A(s)|$被选择，而贪婪动作则以概率 $(1-\varepsilon) \quad +\varepsilon/|A(s)|$被选择。

接下来我们将证明，该策略在所有 ε -软策略中是最优的。

定义符号集：

- $\mathcal{S}$: 状态空间（所有可能状态的集合）。
- $\mathcal{A}(s)$: 在状态 s 下，所有可选动作的集合。
- $|\mathcal{A}(s)|$: 可选动作的总数量。
- $Q^{\pi}(s,a)$: 在**旧策略** π 下的动作价值。
- π': 我们的**新策略**（也就是采用 ϵ-贪心后生成的策略）。
- $\pi'(a|s)$: 在新策略下，状态 s 时选择动作 a 的概率。
- ϵ: 探索率（一个很小的常数，比如 0.1）。

Proof. First, we have for each $s \in \mathcal{S}$:

$$\begin{aligned} Q^{\pi}(s,\pi'(s)) &= \sum_{a\in\mathcal{A}(s)} \pi'(a|s) Q^{\pi}(s,a) \\ &= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_{a\in\mathcal{A}(s)} Q^{\pi}(s,a) + (1-\varepsilon) \max_{a\in cA(s)} Q^{\pi}(s,a). \end{aligned} \tag{5.16}$$

We define $\rho(a|s)$ as

$$\rho(a|s) \triangleq \frac{\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}}{1-\varepsilon}$$

新策略 π' 是一个 ϵ-贪心策略。这就意味着，对于所有的动作，我们先雨露均沾地分给它们 $\frac{\epsilon}{|\mathcal{A}(s)|}$ 的保底随机概率（对应加号左边）。然后，我们把剩下 $(1-\epsilon)$ 的绝对大头概率，全部押宝在那个 Q 值最大的“学霸”动作上（对应加号右边）。

对于定义，旧策略 π 也是一个带有 ϵ 随机性的软策略。作者在这里变了个魔术，强行把旧策略里那部分“雨露均沾的 ϵ 随机杂音”给抠掉了，提取出了旧策略中真正“认真做选择”的概率分布，并把它命名为 $\rho(a|s)$。

Note that π is ε-soft, hence $\pi(a|s) \geq \varepsilon/|\mathcal{A}(s)|$, and thus $\rho(a|s)$ are non-negative. Moreover, we have

$$\begin{aligned}\sum_{a\in\mathcal{A}(s)} \rho(a|s) &= \frac{1}{1-\varepsilon} \sum_{a\in\mathcal{A}(s)} \left(\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}\right) \\ &= \frac{1}{1-\varepsilon} \left(\sum_{a\in\mathcal{A}(s)} \pi(a|s) - \frac{|\mathcal{A}(s)|}{|\mathcal{A}(s)|}\varepsilon\right) \\ &= \frac{1}{\varepsilon}(1-\varepsilon) = 1. \end{aligned} \tag{5.17}$$

这一长串推导只是为了证明一件事：把所有的 $\rho(a|s)$ 加起来正好等于 1。这说明 $\rho(a|s)$ 确实是一个合法有效的概率分布

Hence, we may consider $\rho(a|s)$ as a probability distribution over $\mathcal{A}(s)$, and thus

$$\max_{a\in\mathcal{A}(s)} Q^\pi(s,a) \geq \sum_{a\in\mathcal{A}(s)} \rho(a|s) Q^\pi(s,a). \tag{5.18}$$

左边是所有动作中的**最高分**（最大 Q 值）。右边是用 $\rho(a|s)$ 这个概率分布算出来的**加权平均分**。一个集合里的最大值，永远大于或等于它的任何加权平均值。

现在，把【公式 5.18】里那个更小的“加权平均分”，替换掉【公式 5.16】右边的“最高分”：

$$Q^\pi(s,\pi'(s)) \geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + \sum_a (1-\epsilon) \left(\frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon}\right) Q^\pi(s,a)$$

把后面那一团化简，$(1-\epsilon)$ 约掉：

$$= \sum_a \left[\frac{\epsilon}{|\mathcal{A}(s)|} + \pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}\right] Q^\pi(s,a)$$

一正一负抵消：

$$= \sum_{a\in\mathcal{A}(s)} \pi(a|s) Q^\pi(s,a) = V^\pi(s)$$

最后得出：

$$Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$$

这意味着，在状态 s 按照我们的**新策略 π'（ϵ-贪心）走一步，其预期的回报绝对大于或等于**一直遵循旧策略 π 的总回报 $V^{\pi}(s)$。策略得到了实质性的提升！

也就是假设当前状态有 $|A(s)|$ 个可选动作（比如上下左右 4 个）。我们设定一个很小的概率 ϵ（比如 0.1）。

1. **抛硬币决定：**
   - 有 $1-\epsilon$ 的超大几率（90%），我们按原计划行事，选择当前 Q 值最高的**贪心动作**。
   - 有 ϵ 的极小几率（10%），我们决定"瞎试一次"，在所有可用的动作里**完全随机**挑一个。
2. **最终每个动作被选中的数学概率：**
   - **非贪心动作（那些原本不会被选的动作）：** 它们只能靠那 ϵ 的随机几率被抽中，所以概率是 $\frac{\epsilon}{|A(s)|}$。
   - **贪心动作（那个最优动作）：** 它不仅独占了 $1-\epsilon$ 的必选几率，同时在那 ϵ 的随机池子里，它也有可能被瞎猫碰上死耗子抽中。所以总概率是：$1-\epsilon+\frac{\epsilon}{|A(s)|}$

Algorithm 5.2 On-policy Monte Carlo control without exploring starts

for each $s \in \mathcal{S}$ **do** ▷ Initialization
  Initialize $C(s,a) = 0,\ Q(s,a) \in \mathbb{R},\ \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$
  Initialize $\pi(s)$ to be a ε-soft policy
end for
for each episode **do**
  Choose $s_0 \in \mathcal{S},\ a_0 \in \mathcal{A}(s)$ randomly ▷ Exploring starts
  Sample an episode following π: $(s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T)$
  Let $g_T \leftarrow 0$
  for each $t = T-1, T-2, \ldots, 0$ **do**
    Let $g_t \leftarrow \gamma g_{t+1} + r_{t+1}$
    Let $C(s_t, a_t) \leftarrow C(s_t, a_t) + 1$
    Let $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{C(s_t,a_t)}(g_t - Q(s_t, a_t))$ ▷ Update value
    Let $a^* \leftarrow \operatorname{argmax}_a Q(s_t, a)$
    for each $a \in \mathcal{A}(s_t)$ **do**
      $\pi(a|s_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s_t)| & (a = a^*) \\ \epsilon/|\mathcal{A}(s_t)| & (a \neq a^*) \end{cases}$ ▷ Update policy
    end for
  end for
end for
return π and Q

真正的区别全在**最后一部分（策略是如何更新的）**：在 上一个算法（探索启动）里，策略 $\pi(s)$ 存的是一个具体的动作（比如“向左走”）。在这里，策略 $\pi(a|s)$ 变成了一个**概率分布**。

最后那个 `for each a` 循环：

1. **找“最优”：** 它先通过 `argmax` 找出当前状态下 Q 值最高的最优动作 a^*。
2. **分发概率：**
   - 如果是最优动作 a^*，就分给它极大的概率：$1 - \epsilon + \epsilon/|\mathcal{A}(s_t)|$。
   - 如果不是最优动作（备胎），就只分给它一点点“用来探索”的概率：$\epsilon/|\mathcal{A}(s_t)|$。

▼ 例子：

一、 到底什么是 Episode（回合 / 幕）？

Episode（中文常翻译为“回合”或“一幕”）可以简单理解为**“完整的一局游戏”**或者“一个完整的任务周期”**。

在强化学习中，一个 Episode 必须满足一个硬性条件：**它必须有一个明确的起点，和一个明确的终点（Terminal State）。**

- **打游戏的例子：** 在玩《空洞骑士》时，你走进 Boss 房间是起点，你把 Boss 打败了或者你阵亡了，屏幕暗下来弹出提示，这就是一个明确的终点。从进门

到结束的这完整的一次战斗，就是一个 Episode。

- **音乐制作的例子：** 从你在 Logic Pro 里新建一个空白工程（起点），到你经过编曲、录音、混音，最终导出一首完整的 J-Pop 歌曲（终点），这整个创作过程就是一个 Episode。

二、 为什么蒙特卡洛（MC）必须“每次 Episode 结束后”才更新状态 S？

这和蒙特卡洛算法的核心思想有关——它本质上是一个**“事后诸葛亮”**。

原因：不走到最后，你根本不知道之前的决定是对是错。

在蒙特卡洛方法里，我们评估一个状态 S 好不好，看的是**“从这个状态开始，直到游戏结束，我一共能拿到多少总分（也就是之前推导的累计回报 G_t）”。
既然你要算“直到游戏结束”的总分，你就必须耐心地等这局游戏完全打完**。

反面教材（如果走一步就更新会怎样？）：

假设你在下象棋，你走了一步“弃车保帅”（状态 S），这步棋刚走完，系统立刻扣了你 100 分（因为你损失了一个车）。如果你不看结尾就立刻更新，你会觉得“状态 S 简直糟透了”。

但实际上，正是因为这步“弃车”，你五步之后直接把对面将死了，赢得了整局游戏（+10000分）。

蒙特卡洛方法不会目光短浅地看眼前的得失。它会等你这局棋彻底下完（Episode 结束），拿着最终的胜利，倒退回去对“弃车保帅”那个状态 S 说：“兄弟，虽然当时看你亏了，但最终证明你是个极其优秀的策略！”然后给它的价值狠狠加分。

三、 现实案例解题，帮你彻底弄懂

我们用两个具体的案例，来看看蒙特卡洛方法是怎么在 Episode 结束后算账的。

案例一：量化交易中的策略评估

背景： 你写了一个自动交易程序，每天早上 9 点半开盘开始交易，下午 4 点收盘结算。

定义：

- **Episode：** 完整的一个交易日（从 9:30 到 16:00）。
- **状态 S_1：** 上午 10:00，系统检测到“某股票 MACD 出现金叉”。
- **动作 A：** 全仓买入。

问题：蒙特卡洛算法是如何评估“上午 10:00 出现金叉（S_1）”这个状态的价值的？

解答过程：

1. **执行动作：** 程序在 10:00 识别到 S_1，执行了“全仓买入”。
2. **耐心等待：** 10:01 分，股票跌了，账面亏损。**此时蒙特卡洛算法什么都不做，它不更新 S_1 的价值，继续等。**
3. **Episode 结束：** 下午 4:00 闭市，今天交易结束。结算发现，虽然上午跌了，但下午暴涨，今天总盈利是 5000 元（最终的回报 G_t）。
4. **秋后算账（更新 S）：** 算法拿着这 5000 元，顺着时间线倒推回去，找到了上午 10:00 那个状态 S_1。它记录下来：在状态 S_1 下买入，最终能赚 5000。于是它提高 S_1 的价值分数。如果有 100 个交易日（100 个 Episodes），它就把这 100 次的最终盈亏求个平均，作为 S_1 最终的评估价值。

案例二：训练 AI 玩迷宫游戏

背景： AI 在一个迷宫里，要找到出口。每走一步扣 1 分（为了鼓励它走捷径），找到出口加 100 分。

定义：

- **Episode：** 从起点出发，直到找到出口（终点）。
- **状态 $S_{岔路口}$：** AI 走到了迷宫中间的一个十字路口。

问题：AI 在十字路口向左走进了死胡同，退出来后又向右走最终找到了出口。蒙特卡洛怎么更新 $S_{岔路口}$ 的价值？

解答过程：

1. **收集轨迹：** AI 玩完了一整局（Episode 结束）。它经历的路线是：起点 → … → $S_{岔路口}$ → 左转进死胡同（扣分） → 退回 $S_{岔路口}$ → 右转 → … → 出口（加 100 分）。这整局它一共走了 50 步，最终得分是 $100 - 50 = 50$ 分。
2. **更新价值：** 蒙特卡洛算法开始回顾这局游戏。
   - 如果用 **First-visit MC（首次访问）**：它只看 AI **第一次**到达 $S_{岔路口}$ 后的总得分。第一次到这里后，AI 走错路又走对路，总共花了 50 分钟拿到奖励，最终收益是 50 分。它就把 50 记入 $S_{岔路口}$ 的账本。

- 如果用 **Every-visit MC（每次访问）**：AI 第二次回到 $S_{岔路口}$ 时（从死胡同出来），离出口已经很近了。假设从第二次访问算起，它只走了 10 步就出去了，后续收益是 $100 - 10 = 90$ 分。那么算法会把 50 和 90 都记录在案，用来求平均。

- 1. **G_t (Return，累计回报)**：
   - **通俗理解：** **"这局游戏"**的真实结算单。
   - **定义：** 它是在**单次**具体的 Episode 中，从时刻 t 开始直到结束，你实际拿到手的总奖励（打了折扣 γ）。每一次玩，因为过程不同，G_t 都可能不一样。
   2. **$V(s)$ (State-Value Function，状态价值函数)**：
   - **通俗理解：** 站在当前位置，**"预测"**未来能拿多少分。
   - **定义：** 它是无数次 G_t 的**数学期望（平均值）**。比如在吃鸡游戏里，"身处决赛圈且满血满防"这个状态的 $V(s)$ 极高，因为从大数据来看，处于这个状态的玩家吃鸡率（平均回报）极高。
   3. **$Q(s, a)$ (Action-Value Function，动作价值函数)**：
   - **通俗理解：** 站在当前位置，**"做某个具体动作"**能拿多少分。
   - **定义：** 它比 $V(s)$ 更进一步。如果 $V(s)$ 是问"我在决赛圈好不好？"，那 $Q(s, a)$ 就是问"我在决赛圈（s），现在选择扔手雷（a），未来的平均收益是多少？"。**蒙特卡洛控制的核心就是算这个 Q 表。**
   4. **π (Policy，策略)**：
   - **通俗理解：** 智能体的**"大脑决策说明书"**。
   - **定义：** 到了某个状态 s，我该选哪个动作 a，全听 π 的指挥。

二、 π 产出的到底是什么？0-1 的概率还是分布？

这是一个极好的问题。**π 产出的是一个概率分布 (Probability Distribution)**。

$\pi(a|s)$ 的数学含义是：在状态 s 下，选择动作 a 的概率。所有可选动作的概率加起来必须等于 1。

这个概率分布是怎么得出来的？

这完全取决于你的算法怎么设计这个"大脑"：

• 情况 A：确定性策略 (Deterministic Policy)

就像一个极其固执的人。比如在路口（s），他认定"向右转"的 Q 值最高。

此时 π 输出的分布就是：`[左: 0, 右: 1.0, 前: 0, 后: 0]`。这就是 0-1 概率。

• 情况 B：ϵ-贪心策略 (ϵ-greedy, 即软策略)

这就是我们在"不带探索性出发"里用的方法。假设在路口，经过计算"向右转"的分数依然最高。我们设定探索率 $\epsilon = 0.1$，总共有 4 个动作。

按照公式，我们会把大部分概率给“最优动作”，剩下的概率平分给“所有动作（包括最优动作自己）”。
此时 π 输出的分布就会变成这样：

- 向左（备胎）：$0.1 / 4 = 0.025$
- 向前（备胎）：$0.1 / 4 = 0.025$
- 向后（备胎）：$0.1 / 4 = 0.025$
- **向右（最优）：$(1 - 0.1) + (0.1 / 4) = 0.925$**

你看，这产出的就是一个**连续的概率分布** `[0.025, 0.925, 0.025, 0.025]`。每次走到路口，代码就会根据这个概率权重去“抽卡”决定下一步怎么走。

三、 现实案例：探索性出发 vs 非探索性出发

为什么我们要废这么大劲去搞 ϵ-贪心？看看下面这两个场景你就懂了。

1. 探索性出发 (Exploring Starts) 的现实场景：Unity 多智能体仿真

假设你在用 Unity 引擎开发一个多智能体追逐游戏的 AI。
因为你掌握着底层代码（拥有“上帝视角”），你可以写一个脚本：**每次 Episode 重新开始时，强制把智能体生成 (Spawn) 在地图的任意随机坐标，并且强制它开局第一步执行任意动作。**

- **优点：** 就算 AI 目前是个只会按“前进”的傻子，通过这种“随机传送”的开局，它也能被强制体验到地图的每一个角落（遍历所有状态）。
- **局限：** 必须在虚拟的、可以完全操控环境的模拟器里才能实现。

2. 非探索性出发 (Without Exploring Starts) 的现实场景：量化交易平台

现在，你想训练一个自动化量化交易的 AI。
你的环境是真实的沪深股市或加密货币市场。你**绝对不可能**命令纳斯达克交易所：“喂，麻烦你们把今天的开盘状态随机重置到 2008 年金融危机时的点位，我想让我的 AI 探索一下在那种状态下满仓做多的感觉。”

- **现实：** 股市每天怎么开盘，你只能被动接受。你无法“随机设定开局状态”。
- **解法：** 既然不能靠“出生在罗马”来探索世界，AI 就只能在每天的正常交易中，靠着那 $\epsilon = 0.1$ 的好奇心（ϵ-贪心策略，即输出刚才那种 `[0.025, 0.925, ...]` 的概率分布），偶尔买点冷门股、偶尔试着做空，靠自己一步步试错来探索未知的状态。

四、 灵魂拷问：为什么数学证明里要强行定义一个 $\rho(a|s)$？

这个 $\rho(a|s)$ 是一个**纯粹为了写论文证明而发明的“数学手术刀”**，你在写代码的时候完全不需要用到它。

它的存在是为了解决一个逻辑死结：

我们使用了 ϵ-贪心策略（新策略 π'），也就是说 AI 有 10% 的时间在瞎玩。
我们要向别人证明：**即使 AI 有 10% 的时间在瞎玩，它的新策略也一定比旧策略 π 更好（赚得更多）。**

但是，旧策略 π 本身也是一个带有 10% 瞎玩概率的策略！拿着一堆带有随机噪音的东西去比大小，数学上很难算。

怎么证明？

作者定义 $\rho(a|s)$，本质上是做了一个“提纯”手术。

1. 他用公式把旧策略 π 里面那 10% 的“随机瞎玩概率”给**精准切除**了。
2. 剩下的那个纯粹代表“认真做选择”的概率分布，就叫做 $\rho(a|s)$。
3. 然后作者用数学不等式证明：哪怕是新策略里的“最高分”（不管那些瞎玩的），也绝对大于旧策略里那些“认真做选择”的加权平均分。
4. 一顿代数抵消之后，完美证明了：新策略 $\ge$ 旧策略。

简而言之，$\rho(a|s)$ 是为了在数学推导中**剥离探索带来的随机噪音，裸露出策略的真实意图**，从而完成不等式的证明。